
🔵 **Core Fundamentals**

- What are Microservices & why use them (vs Monolith)
- Microservices vs SOA (Service Oriented Architecture)
- Characteristics of Microservices
- Advantages & Disadvantages
- When NOT to use Microservices

🔵 **Design Principles**

- Single Responsibility Principle
- Domain Driven Design (DDD) basics
- Bounded Context
- Loose Coupling & High Cohesion
- API First Design

🔵 **Communication Patterns**

- Synchronous communication (REST, gRPC)
- Asynchronous communication (Kafka, RabbitMQ)
- Request-Response vs Event Driven
- Choreography vs Orchestration
- Synchronous vs Asynchronous — when to use which

🔵 **API Gateway**

- What is an API Gateway & why
- Responsibilities (routing, auth, rate limiting, load balancing)
- Spring Cloud Gateway
- Netflix Zuul
- Gateway vs Load Balancer

🔵 **Service Discovery**

- Why Service Discovery?

- Client-side vs Server-side discovery
- Eureka (Netflix OSS)
- Consul
- Kubernetes-native discovery

🔵 **Inter-Service Communication**

- REST with RestTemplate & WebClient
- OpenFeign (Declarative REST client)
- gRPC basics
- Synchronous call chain problems
- Timeouts & connection pooling

🔵 **Resilience & Fault Tolerance**

- Circuit Breaker pattern (Resilience4j)
- Retry pattern
- Fallback pattern
- Bulkhead pattern
- Rate Limiter pattern
- Timeout pattern
- Spring Cloud Circuit Breaker

🔵 **Configuration Management**

- Why centralized config?
- Spring Cloud Config Server
- Config Server with Git backend
- Refreshing config at runtime (@RefreshScope)
- Vault for secrets management

🔵 **Event Driven Architecture**

- Event Driven vs Request Driven
- Domain Events

- Event Sourcing
- CQRS (Command Query Responsibility Segregation)
- Eventual Consistency
- Kafka as event backbone

🔵 **Saga Pattern (Distributed Transactions)**

- Why distributed transactions are hard
- Two-Phase Commit (2PC) problems
- Saga pattern — Choreography based
- Saga pattern — Orchestration based
- Compensating transactions
- Outbox pattern

🔵 **Data Management**

- Database per service pattern
- Shared database anti-pattern
- Polyglot persistence
- Data consistency challenges
- Eventual consistency vs Strong consistency

🔵 **Security**

- Authentication & Authorization in Microservices
- JWT (JSON Web Tokens)
- OAuth 2.0 & OpenID Connect
- API Gateway security
- Service-to-service security (mTLS)
- Keycloak basics

🔵 **Observability & Monitoring**

- The 3 pillars — Logs, Metrics, Traces
- Distributed Tracing (Zipkin, Jaeger)

- Spring Cloud Sleuth / Micrometer Tracing
- Correlation ID & Trace ID
- Centralized Logging (ELK Stack)
- Metrics (Prometheus + Grafana)
- Health checks & Actuator

🔵 **Containerization & Deployment**

- Docker basics for Microservices
- Docker Compose for local development
- Kubernetes basics (Pods, Services, Deployments)
- ConfigMaps & Secrets in Kubernetes
- Service Mesh (Istio basics)
- CI/CD pipelines for Microservices

🔵 **Testing Microservices**

- Unit testing individual services
- Integration testing
- Contract testing (Pact)
- End-to-end testing challenges
- Consumer Driven Contract testing

🔵 **Patterns & Best Practices**

- Strangler Fig pattern (Monolith to Microservices migration)
- Anti-Corruption Layer
- Sidecar pattern
- Ambassador pattern
- Backend for Frontend (BFF) pattern
- Idempotency in APIs

🔵 **Spring Boot & Spring Cloud (Java Specific)**

- Spring Boot for Microservices

- Spring Cloud Netflix (Eureka, Ribbon)
- Spring Cloud Gateway
- Spring Cloud Config
- Spring Cloud OpenFeign
- Spring Cloud Circuit Breaker (Resilience4j)
- Spring Cloud Stream (Kafka/RabbitMQ abstraction)

🔵 **Scenario Based Questions (Must Prep)**

- How do you handle distributed transactions?
- How do you ensure data consistency across services?
- What happens if a downstream service is down?
- How do you debug an issue across multiple services?
- How do you migrate a monolith to microservices?
- How do you handle versioning of APIs?
- How do you secure inter-service communication?
- How do you handle cascading failures?

🔵 **Topic 1: Monolith vs Microservices**

What is a Monolith?

A monolith is a **single deployable unit** where all features of an application are bundled together.

Monolith Application (single JAR/WAR):

├— User Module

├— Order Module

├— Payment Module

├— Inventory Module

└— Notification Module

Everything runs in **one process**, deployed **together**.

What are Microservices?

Microservices is an architecture where the application is split into **small, independent services**, each:

- Running in its **own process**
- Having its **own database**
- Deployable **independently**
- Communicating via **APIs or events**

Microservices:

├— User Service (port 8081, own DB)

├— Order Service (port 8082, own DB)

├— Payment Service (port 8083, own DB)

├— Inventory Service (port 8084, own DB)

└— Notification Service(port 8085, own DB)

Monolith vs Microservices

Aspect	**Monolith**	**Microservices**
Deployment	Single unit	Independent per service
Scaling	Scale entire app	Scale only needed service
Technology	Single tech stack	Each service can use different tech
Development	Simple initially	Complex but flexible

Aspect	Monolith	Microservices
Failure Impact	Entire app goes down	Only that service fails
Team Size	Small teams	Large teams (one team per service)
Database	Single shared DB	Database per service
Performance	Faster (in-process calls)	Slower (network calls)

Real World Analogy

Monolith = A Swiss Army knife — everything in one tool, simple but limited
Microservices = A toolbox — separate specialized tools, more powerful but more to manage

---

When to use Microservices?

✅ Large team (10+ developers)
✅ Different modules need different scaling
✅ High availability requirement
✅ Multiple teams working independently

When NOT to use Microservices?

❌ Small team or startup
❌ Simple application
❌ Tight deadline
❌ Limited DevOps maturity

Interview Answer (30 seconds)

"Microservices break a large application into small, independently deployable services each with their own database. Unlike monoliths where everything is tightly coupled, microservices allow independent scaling, deployment, and development. However they introduce complexity in communication, data consistency, and operations."

🔵 Topic 2: Core Characteristics of Microservices

---

7 Key Characteristics

1. Single Responsibility
Each service does **one thing and does it well**

✅ Order Service → only handles orders

❌ Order Service → handles orders + payments + notifications

2. Independent Deployment
Each service can be deployed without affecting others

Deploy Payment Service v2.0

→ Order Service still running v1.0 ✅ (not affected)

3. Database per Service
Each service owns its data — no shared databases

Order Service → MySQL

Payment Service → PostgreSQL

Product Service → MongoDB

4. Loose Coupling
Services are independent — change in one doesn't break another

Payment Service changes internally

→ Order Service doesn't need to change ✅

5. High Cohesion
Everything related to one business capability stays together

Order Service contains:

├── Order creation logic

├── Order repository

├── Order events

└── Order APIs

(everything order-related in one place)

6. Communication via APIs or Events
Services talk to each other only through well-defined interfaces

Synchronous: Order Service → REST API → Payment Service

Asynchronous: Order Service → Kafka Event → Notification Service

7. Decentralized Data Management
No single database for all services — each manages its own data store

---

🔵 Topic 3: REST Communication + OpenFeign

How do Microservices communicate?

The most common way is **REST over HTTP**.

Option 1: RestTemplate (Old way)

java

```
@Service
public class OrderService {

    private RestTemplate restTemplate;

    public PaymentResponse processPayment(String orderId) {
        String url = "http://payment-service/api/payments/" + orderId;
        return restTemplate.getForObject(url, PaymentResponse.class);
    }
}
```

Problems: verbose, manual URL building, no load balancing built-in.

Option 2: WebClient (Reactive, modern)

java

```
WebClient webClient = WebClient.create("http://payment-service");

PaymentResponse response = webClient.get()
    .uri("/api/payments/{orderId}", orderId)
    .retrieve()
    .bodyToMono(PaymentResponse.class)
    .block();
```

Option 3: OpenFeign ⭐ (Most popular, interview favourite)

OpenFeign lets you write REST clients as **simple Java interfaces** — no boilerplate code.

java

```
// Step 1: Add dependency
// spring-cloud-starter-openfeign
```

```
// Step 2: Enable Feign
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication { }

// Step 3: Define Feign Client interface
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @GetMapping("/api/payments/{orderId}")
    PaymentResponse getPayment(@PathVariable String orderId);

    @PostMapping("/api/payments")
    PaymentResponse createPayment(@RequestBody PaymentRequest request);
}

// Step 4: Use it like any Spring bean
@Service
public class OrderService {

    @Autowired
    private PaymentClient paymentClient;

    public void placeOrder(Order order) {
        // Just call it like a normal method!
        PaymentResponse payment = paymentClient.createPayment(
            new PaymentRequest(order.getId(), order.getAmount())
        );
    }
}
```

Why OpenFeign?

✅ No boilerplate code
✅ Integrates with Eureka (service discovery) automatically
✅ Integrates with Resilience4j (circuit breaker) easily
✅ Clean and readable code

🔵 **Topic 4: API Gateway**

What is an API Gateway?

An API Gateway is the **single entry point** for all client requests to your microservices.

Without Gateway:

Client → directly calls each service

Client needs to know: 8081, 8082, 8083, 8084... 😰

With Gateway:

Client → API Gateway (8080) → routes to correct service ✅

Client only needs to know ONE address

What does API Gateway do?

Responsibility	Details
Routing	Routes requests to correct microservice
Authentication	Validates JWT tokens centrally
Rate Limiting	Limits requests per client
Load Balancing	Distributes load across service instances
SSL Termination	Handles HTTPS centrally
Logging	Centralized request logging
Request/Response transformation	Modify headers, payloads

Spring Cloud Gateway Example

yaml

application.yml

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service      # lb = load balanced via Eureka
          predicates:
            - Path=/api/orders/**      # route if path matches
          filters:
            - StripPrefix=1            # remove /api prefix before forwarding

        - id: payment-service
          uri: lb://payment-service
          predicates:
            - Path=/api/payments/**
```

Request: GET /api/orders/123

Gateway routes to → order-service/orders/123 ✅

Request: POST /api/payments

Gateway routes to → payment-service/payments ✅

Interview Answer

"API Gateway acts as the single entry point for all client requests. It handles cross-cutting concerns like authentication, rate limiting, routing, and logging centrally — so individual services don't have to implement these themselves."

🔵 Topic 5: Service Discovery (Eureka)

The Problem

In microservices, services are dynamic — they can:

- Scale up (multiple instances)

- Change IP addresses
- Go down and come back

How does Order Service know where Payment Service is?

Payment Service instance 1: 192.168.1.10:8083

Payment Service instance 2: 192.168.1.11:8083 (new instance scaled up)

Payment Service instance 3: 192.168.1.12:8083 (another instance)

Hardcoding IPs is impossible! ❌

Solution: Service Discovery

A **Service Registry** keeps track of all running service instances.

Eureka (Netflix OSS)

Eureka is the most popular service registry in Spring ecosystem.

How it works:

1. Each service registers itself with Eureka on startup

   "Hi Eureka! I am payment-service, running at 192.168.1.10:8083"

2. Eureka maintains a registry of all services

   Registry:

   ├── order-service   → [instance1, instance2]

   ├── payment-service → [instance1, instance2, instance3]

   └── user-service   → [instance1]

3. When Order Service wants to call Payment Service:

   "Eureka, give me address of payment-service"

   Eureka returns one instance → Order Service calls it ✅

4. Services send heartbeats every 30 seconds

   If no heartbeat → Eureka removes that instance

Spring Boot Implementation

java

```
// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication { }
```

yaml

```
# Each microservice application.yml
spring:
  application:
    name: payment-service  # service name in registry

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

java

```
// Each microservice main class
@SpringBootApplication
@EnableEurekaClient
public class PaymentServiceApplication { }
```

🔵 Topic 6: Circuit Breaker (Resilience4j)

The Problem: Cascading Failures

Order Service → calls → Payment Service (which is DOWN 💥)

Without Circuit Breaker:

Order Service keeps calling Payment Service

→ Requests pile up, threads exhausted

→ Order Service also goes DOWN 💥

→ Other services depending on Order Service go DOWN 💥

→ Entire system collapses 😱 (Cascading failure)

Solution: Circuit Breaker Pattern

Just like an electrical circuit breaker — **stops the flow when something goes wrong**.

3 States of Circuit Breaker:

CLOSED (Normal) → requests flow through

↓ (failures exceed threshold)

OPEN (Tripped) → requests blocked, fallback returned immediately

↓ (after wait time)

HALF-OPEN (Testing) → allows few requests to test if service recovered

↓ (if success)

CLOSED again ✅

Resilience4j Implementation

java

```
// application.yml
resilience4j:
  circuitbreaker:
    instances:
      payment-service:
        sliding-window-size: 10          # last 10 requests
        failure-rate-threshold: 50       # open if 50% fail
        wait-duration-in-open-state: 10s # wait 10s before half-open
        permitted-calls-in-half-open-state: 3
```

java

```
@Service
public class OrderService {
```

```
    @Autowired
    private PaymentClient paymentClient;

    // Circuit Breaker + Fallback
    @CircuitBreaker(name = "payment-service", fallbackMethod = "paymentFallback")
    public PaymentResponse processPayment(PaymentRequest request) {
        return paymentClient.createPayment(request);
    }

    // Fallback method — called when circuit is OPEN
    public PaymentResponse paymentFallback(PaymentRequest request, Exception ex) {
        // Return default response or queue for retry
        return new PaymentResponse("PENDING", "Payment service unavailable, will retry");
    }
}
```

Other Resilience4j Patterns

```
java
// Retry — retry failed calls automatically
@Retry(name = "payment-service", fallbackMethod = "paymentFallback")

// Rate Limiter — limit calls per time period
@RateLimiter(name = "payment-service")

// Bulkhead — limit concurrent calls
@Bulkhead(name = "payment-service")

// Timeout — fail fast if takes too long
@TimeLimiter(name = "payment-service")
```

Interview Answer

"Circuit Breaker prevents cascading failures. When a downstream service starts failing beyond a threshold, the circuit opens and immediately returns a fallback response instead of waiting — protecting the calling service from resource exhaustion. After a wait period it tests if the service recovered."

🔵 **Topic 7: JWT + OAuth 2.0**

The Problem

User logs into Order Service ✅

User then calls Payment Service — how does Payment Service

know the user is authenticated? 🤔

Solution: JWT (JSON Web Token)

JWT is a **self-contained token** that carries user information.

JWT Structure (3 parts separated by dots):

eyJhbGciOiJIUzI1NiJ9. ← Header (algorithm)

eyJ1c2VySWQiOiIxMjMifQ. ← Payload (user data)

SflKxwRJSMeKKF2QT4fwpMeJf36. ← Signature (verification)

Payload contains:

json

```
{
  "userId": "123",
  "email": "user@example.com",
  "roles": ["USER", "ADMIN"],
  "exp": 1735689600
}
```

How JWT works in Microservices

1. User logs in → Auth Service validates credentials

2. Auth Service generates JWT token → returns to client

3. Client sends JWT in every request header:

Authorization: Bearer eyJhbGci...

4. API Gateway validates JWT ✅

5. Gateway forwards request + user info to microservice

6. Microservice trusts the request (already validated by gateway)

OAuth 2.0 Basics

OAuth 2.0 is an **authorization framework** — defines HOW tokens are issued.

Key Roles:

├── Resource Owner → User

├── Client → Your app (Order Service)

├── Auth Server → Issues tokens (Keycloak, Okta, Auth0)

└── Resource Server → Protected service (Payment Service)

Flow:

User → logs in → Auth Server

Auth Server → issues JWT → User

User → sends JWT → API Gateway

Gateway → validates with Auth Server → forwards request

🔵 **Topic 8: Saga Pattern**

The Problem: Distributed Transactions

Place Order flow:

1. Order Service → create order

2. Payment Service → deduct payment

3. Inventory Service→ reduce stock

4. Notification Service → send email

What if Payment succeeds but Inventory fails? 😱

→ Order created ✅, Payment deducted ✅, Stock not reduced ❌

→ Data inconsistency!

Traditional database transactions (ACID) don't work across services!

Solution: Saga Pattern

Break the transaction into **a sequence of local transactions**, each publishing an event. If one fails → **compensating transactions** undo previous steps.

Type 1: Choreography Saga

Services react to events — **no central coordinator**.

Order Service → publishes "OrderCreated" event

↓

Payment Service → listens → processes payment → publishes "PaymentProcessed"

↓

Inventory Service→ listens → reduces stock → publishes "StockReduced"

↓

Notification → listens → sends email ✅

If Payment FAILS:

Payment Service → publishes "PaymentFailed"

Order Service → listens → cancels order (compensating transaction)

✅ Simple, no central point of failure
❌ Hard to track overall flow, complex error handling

Type 2: Orchestration Saga ⭐ (More popular)

A central **Saga Orchestrator** tells each service what to do.

Saga Orchestrator:

Step 1: "Payment Service, process payment" ✅

Step 2: "Inventory Service, reduce stock" ✅

Step 3: "Notification Service, send email" ✅

→ Done! ✅

If Step 2 fails:

Orchestrator: "Payment Service, REFUND payment" (compensating)

Orchestrator: "Order Service, CANCEL order" (compensating)

✅ Easy to track, centralized error handling
❌ Orchestrator can become a bottleneck

Interview Answer

"Saga pattern manages distributed transactions by breaking them into local transactions with compensating transactions for rollback. Choreography uses events with no central coordinator while Orchestration uses a central saga orchestrator to manage the flow."

🔵 Topic 9: Database per Service

The Pattern

Each microservice has its **own dedicated database** that no other service can access directly.

✅ Correct:

Order Service → own MySQL DB

Payment Service → own PostgreSQL DB

Product Service → own MongoDB

❌ Wrong (Anti-pattern):

All services → shared single database

Why?

- **Loose coupling** — services are truly independent
- **Independent scaling** — scale DB with the service
- **Technology freedom** — use best DB for each use case
- **No single point of failure**

The Challenge

Order Service needs customer name from User Service DB

→ Cannot directly query User Service DB ❌

Solutions:

1. API call → Order Service calls User Service REST API

2. Event → User Service publishes events, Order Service maintains its own copy

3. CQRS → Maintain a read-optimized view in Order Service

🔵 Topic 10: Distributed Tracing

The Problem

User reports: "My order is slow!"

Request goes through:

Gateway → Order Service → Payment Service → Inventory Service → Notification Service

Where is the bottleneck? 😕

How do you debug across 5 services?

Solution: Distributed Tracing

Assign a unique **Trace ID** to every request that flows through all services.

Request comes in → Gateway assigns Trace ID: "abc-123"

Gateway [abc-123, span-1] → 5ms

Order Service [abc-123, span-2] → 20ms

Payment Service [abc-123, span-3] → 800ms ← BOTTLENECK! 🔴

Inventory Service[abc-123, span-4] → 10ms

Notification [abc-123, span-5] → 5ms

Spring Boot Implementation

yaml

Spring Boot 3.x with Micrometer

dependencies:

- micrometer-tracing-bridge-brave

- zipkin-reporter-brave

```
# application.yml
management:
  tracing:
    sampling:
      probability: 1.0  # trace 100% of requests
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

Zipkin UI shows you the full trace visually — exactly where time was spent! 🎯

🎯 Quick Revision — One Liners

Topic	One Line Answer
Microservices	Small independent services each with own DB, deployed separately
API Gateway	Single entry point handling routing, auth, rate limiting
Service Discovery	Registry (Eureka) tracking all service instances dynamically
Circuit Breaker	Stops cascading failures by short-circuiting failing calls
JWT	Self-contained token carrying user info for stateless auth
Saga Pattern	Manages distributed transactions with compensating rollbacks
Database per Service	Each service owns its DB — no sharing allowed
OpenFeign	Declarative REST client using simple Java interfaces
Distributed Tracing	Trace ID tracks a request across all services for debugging

---

💪 Final Tips for Interview

1. **Always give a real world example** — "In my project, we used Circuit Breaker when..."
2. **Know the WHY** — not just what it is, but what problem it solves
3. **Connect topics** — "API Gateway uses Eureka for service discovery and Resilience4j for circuit breaking"

4. **Be honest** — if you don't know deep internals, say "I understand the concept and have used it at a high level"

You now have everything you need to face a Microservices interview confidently in 1-2 days! 🚀

Would you like me to do a **mock interview** with questions and answers on these topics to test your preparation? 🎯

□□□□□

□